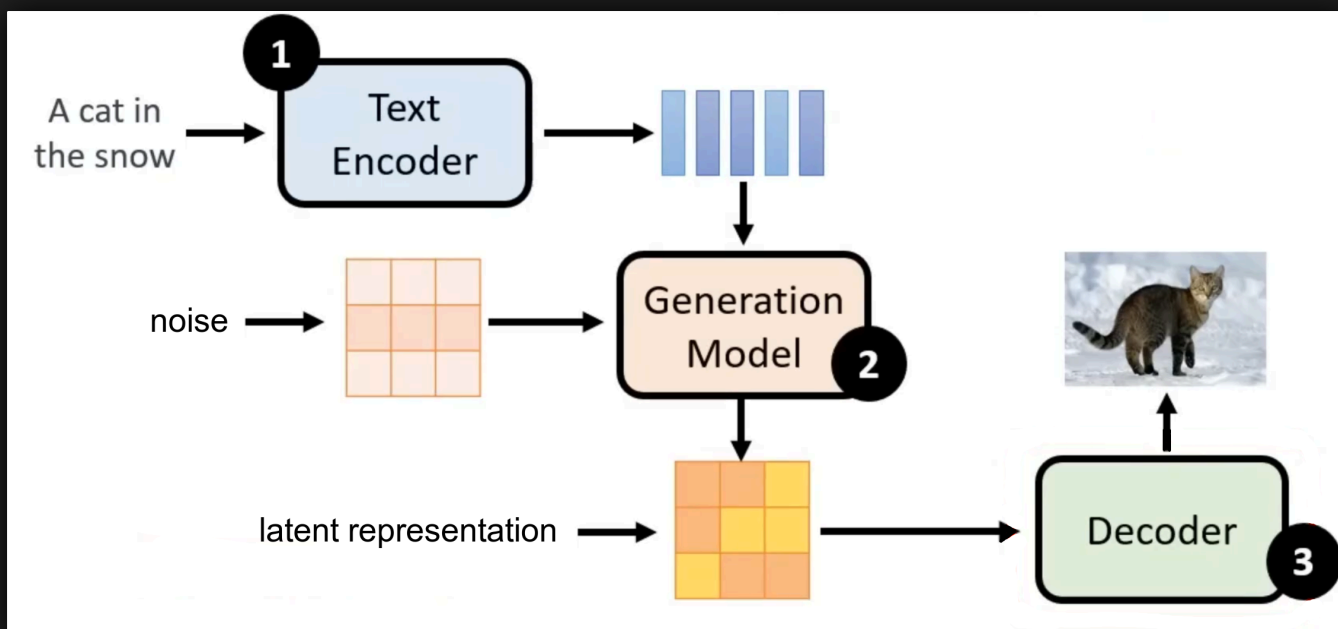


framework



The generation model denoises the initial noise (which is sampled from normal distribution) by multiple iterations, conditioned by the latent context embeddings encoded by the encoder, and ultimately producing the latent representation. The latent representation is then decoded to image.

Note that the whole denoising process is carried out in the latent space. The final latent representation is then be decoded to the pixel space.

mechanism

training:

Algorithm 1 Training

1 : **repeat**

2 : $\mathbf{x}_0 \sim q(\mathbf{x}_0)$

3 : $t \sim \text{Uniform}(\{1, \dots, T\})$

4 : $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

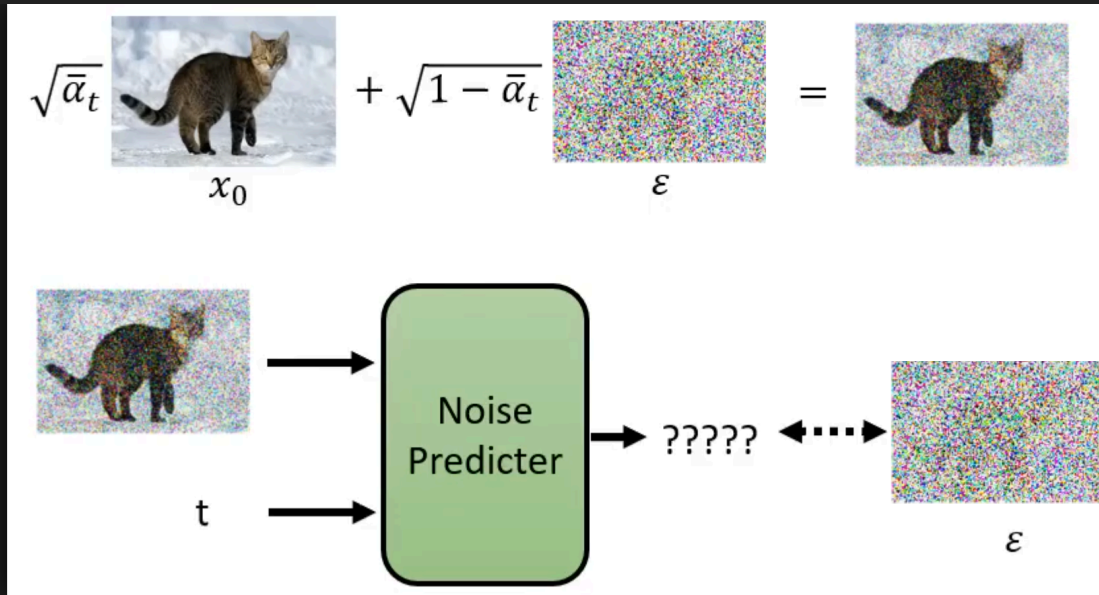
5 : Take gradient descent step on

$$\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta} \left(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t \right) \right\|^2$$

6 : **until** converged

where

- $q(\mathbf{x}_0)$ is the input image distribution.
- $\bar{\alpha}_1, \dots, \bar{\alpha}_T$ is a set of hyperparameters and $\bar{\alpha}_1 < \dots < \bar{\alpha}_T$.
- ϵ_{θ} is the noise predictor.



inference:

Algorithm 2 Sampling

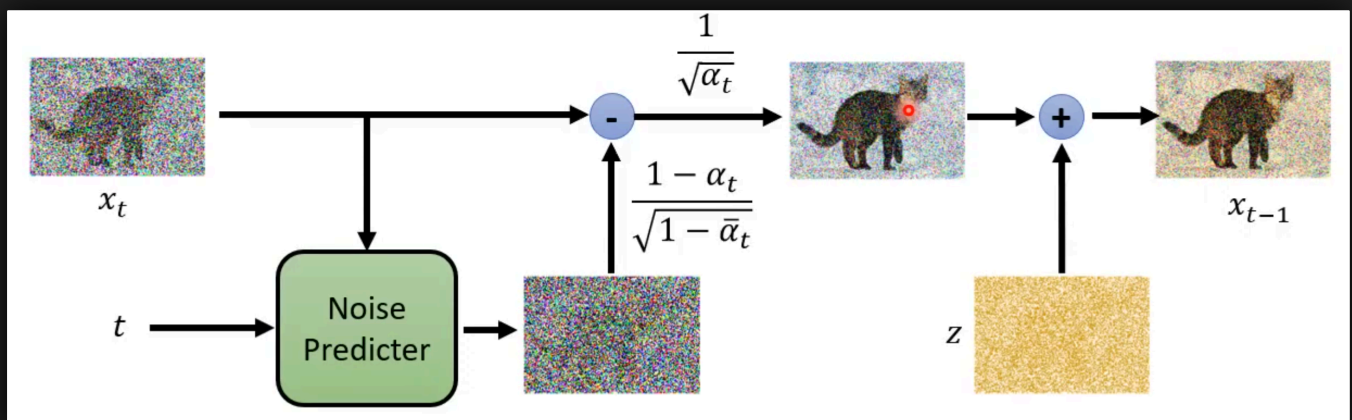
```

1 :  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2 : for  $t = T, \dots, 1$  do
3 :    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4 :    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5 : end for
6 : return  $\mathbf{x}_0$ 

```

where

- β_t is a pre-defined small numerical value (e.g. ranging from 0.0001 to 0.02). It represents the proportion of noise to be newly added to the image at step t .
- $\alpha_t = 1 - \beta_t$ is the *single-step retention*, signifying the signal retained after a single diffusion step.
- $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s = \alpha_1 \times \alpha_2 \times \dots \times \alpha_t$ is the *cumulative retention*, signifying the total signal retained so far at step t .



Why adding $\sigma_t \mathbf{z}$?

- To Enable Generative Diversity

Without this noise, the reverse process becomes deterministic. If you input the same initial random noise (z_T), the math would produce the exact same image every single time. Adding $\sigma_t z$ injects randomness at every step, allowing the model to explore slightly different paths and generate diverse variations of the image.

- To Prevent Blurry Images (Mode Collapse)

The neural network predicts the mean (average) of the data distribution.

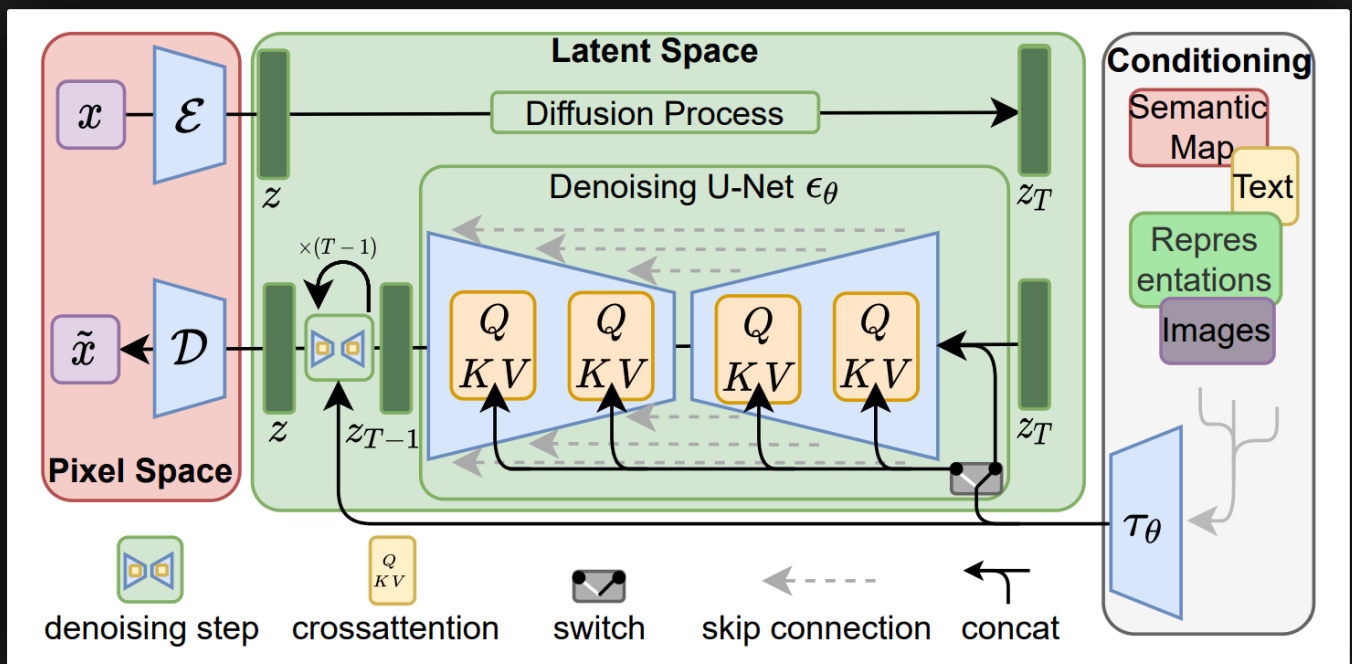
- If you only keep the mean (remove the noise), the model will average out all possible high-frequency details (like hair, fur, or texture), resulting in a smooth, blurry look.
- Adding noise forces the model to move away from the safe "average" and commit to specific, sharp details.
- To Perform True Sampling

Mathematically, the reverse diffusion step is a probability distribution (Gaussian), not a single number.

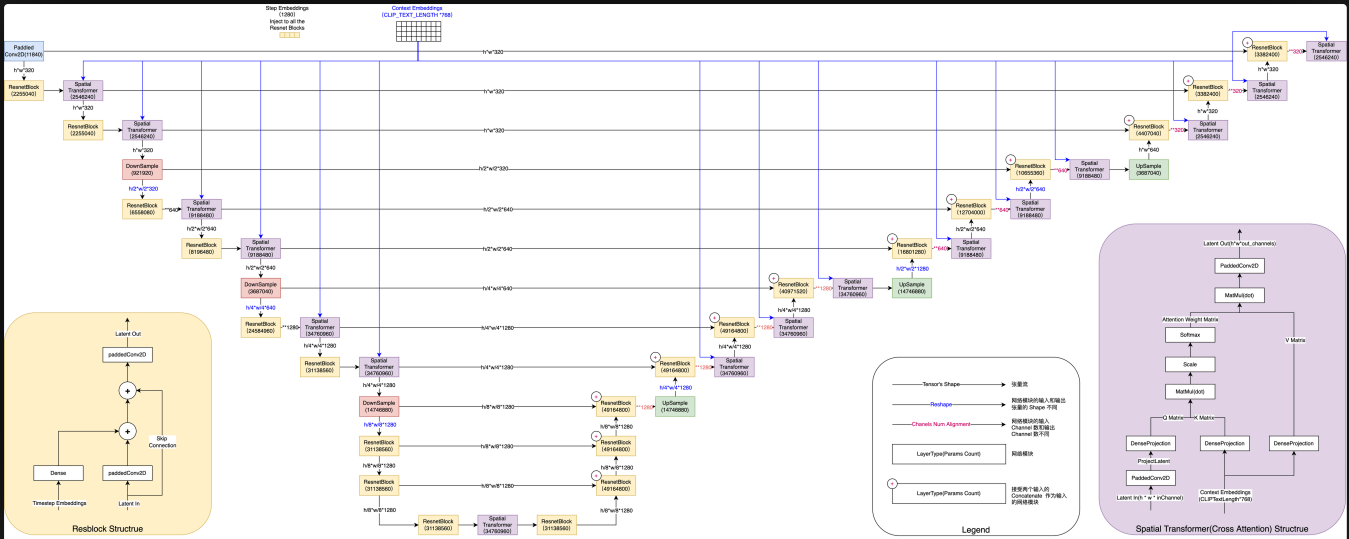
- The formula μ_θ gives us the center of that distribution.
- The term $\sigma_t z$ allows us to mathematically sample a random point from that distribution, rather than just taking the center point.
- In short: It turns the process from a calculation (which outputs a blurry average) into a simulation (which outputs a sharp, specific instance).

Core Idea: Adding autoregressive method (i.e. denoising iteratively in diffusion policies) in non-autoregressive tasks (i.e. image generation in contrast to language generation) to take advantages of both.

Stable Diffusion



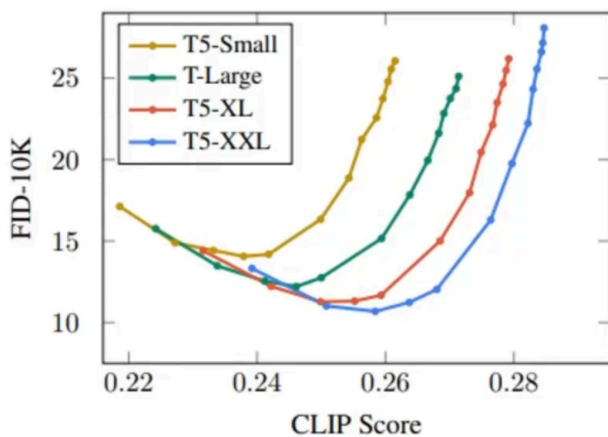
Denoising U-Net ε_θ :



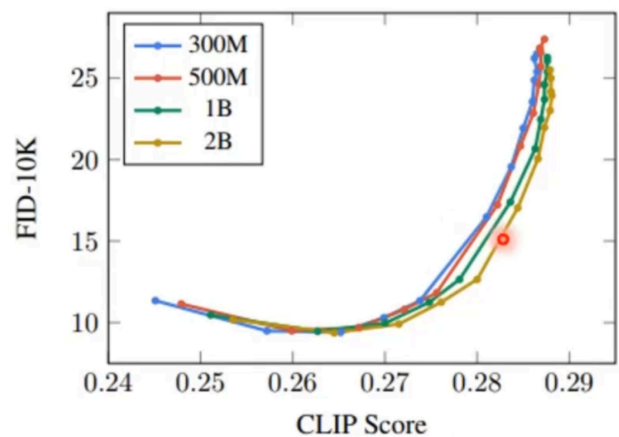
- Dense/DenseProjection: i.e. Linear/LinearProjection
- paddedConv2D: When the kernel scanning an image, edge pixels cannot be fully convolved due to incomplete coverage, resulting in a one-pixel reduction in the output image dimension on all sides (e.g. 64×64 input $\rightarrow 62 \times 62$ output). *Padding* involves appending a border of zeros around the original latent matrix, thus ensuring that the input spatial dimensions match the output spatial dimensions.
- channels: i.e. Dimension of features of each pixel. e.g. for the input image, *channel* = 3 (R, G, B); for the latent matrix, *channel* may increase to 1280. Because for each *ResnetBlock*, the convolution operation is performed by multiple kernels, the *feature maps* produced by each kernel are stacked together, thus resulting extra dimensions added after each convolution. (i.e. $1280 = (\text{num_kernel})_1 \times \dots \times (\text{num_kernel})_n$)

Encoder:

Researches showed that larger encoder size significantly improves model performance, while increasing the size of generation model does not yield a significant improvement in performance.



(a) Impact of encoder size.



(b) Impact of U-Net size.

<https://arxiv.org/abs/2205.11487>